Open Source Software Audit Report
Visual Studio Code

Name: Ashwin Goyal
Reg. no.: 24BCE10596
Course: Open Source Software
Environment: Arch-based Linux (Omarchy)

1. Introduction

Before starting this assignment, I honestly thought open source just meant "free software." But after working on this audit, my understanding changed a lot. Open source is not only about cost — it is about freedom, transparency, and collaboration.

In simple terms, open source allows anyone to see how software works, modify it, and share it further. This creates an environment where development becomes faster because people are not starting from zero every time.

For this report, I have chosen Visual Studio Code. It is one of the most widely used code editors today, and it represents an interesting mix of community-driven development and corporate involvement.

2. Part A — Origin and Philosophy

2.1 Origin Story - Why Visual Studio Code Was Actually Needed

Instead of just mentioning the release year, I tried to understand what problem VS Code was trying to solve.

Earlier, developers didn't really have a balanced option when it came to coding tools. On one side, there were very simple editors like

Notepad++, which were fast but lacked advanced features. On the other side, there were full-scale IDEs like Eclipse or IntelliJ, which were powerful but often heavy and sometimes slow, especially on average systems.

This created a gap.

There was no tool that was both lightweight and feature-rich at the same time.

Visual Studio Code was introduced to fill exactly this gap.

What I found interesting is that it doesn't try to include everything by default. Instead, it follows a modular approach. Users can install extensions depending on their needs. At first, I didn't pay much attention to this, but after exploring it more, I realized this is what makes it so flexible.

For example, a person working in Python can install only Python-related tools, while a web developer can customize it completely differently. So instead of forcing features, it adapts to the user.

Another important factor is cross-platform compatibility. VS Code works almost the same across Linux, Windows, and macOS. This consistency makes it easier for developers to switch between systems without having to relearn tools.

2.2 License Analysis - Ethical Perspective — What I Personally Felt

This section was honestly a bit confusing in the beginning.

Visual Studio Code's open-source core (Code-OSS) uses the MIT License.

At a basic level, it allows:

- free usage
- modification

- distribution

But what I found more important is what the license does *not* require.

Unlike some other licenses, it does not force users to share their modified versions.

This is where it differs from the GPL (General Public License).

GPL is stricter — it ensures that if someone modifies the software, they must also keep it open source.

MIT, on the other hand, is more flexible.

Initially, I thought more freedom automatically means better. But after thinking about it, I realized that it depends on the situation.

Because while MIT encourages wider usage, it also allows companies to use open-source code and not contribute back to the community.

So it becomes a trade-off:

- more freedom → less obligation
- more protection → less flexibility

This made me think of it as a balance between openness and fairness.

2.3 Ethical Perspective — What I Personally Felt

This was probably the most thought-provoking part of the assignment.

At first, open source sounds like an ideal system where everyone contributes and benefits equally. But in reality, things are more complex.

Companies are also involved in open source.

They use open-source projects, build products on top of them, and sometimes make profit from them. This is legally allowed, but it raises questions about fairness.

In the case of Visual Studio Code:

- Microsoft contributes significantly
- but it still controls branding and distribution

So even though the core is open source, the overall system is not fully community-driven.

Earlier, I used to think open source means no control. But now I feel it's more like shared control — sometimes balanced, sometimes not.

This made me realize that open source is not just technical — it also involves ethical and philosophical questions.

3. Part B — Linux Footprint-Observing the Software in a Linux Environment

The investigation was done on an Arch-based Linux system dubbed Omarchy. For this part, I tested everything on an Arch-based Linux system called Omarchy.
This was slightly different from Ubuntu, which I had used earlier. For example, Arch uses the pacman package manager instead of apt, so installation and updates follow a different process.

Key observations:

- Executable location → /usr/bin/code

- Configuration files → ~/.config/Code

One thing I didn't really think about earlier is how applications run in Linux.
VS Code runs as a normal user, not as root.
This is actually important for security because it limits system-level access and reduces the risk of accidental or harmful changes.
Another observation is that VS Code is not a background service.
Unlike servers such as Apache or MySQL, it only runs when the user launches it.
Updates can be managed through:

- pacman (official packages)
- AUR (community-maintained packages)

This shows how open-source software is maintained in a decentralized way, where both official maintainers

4.Part C — FOSS Ecosytem-Understanding the Ecosystem Around VS Code

Initially, I thought VS Code was just a standalone tool. But after looking into it, I realized it depends on multiple other technologies.

Some of the main components include:

- Electron (for building desktop apps)
- Node.js (for runtime environment)
- Chromium (for rendering interface)

So instead of being built from scratch, it is actually a combination of existing tools layered together.

This connects with the idea of "standing on the shoulders of giants," where new software is built on top of previous work.

Another important part is the extension ecosystem.

This is probably one of the biggest strengths of VS Code.

You can install extensions for:

- programming languages
- debugging tools
- themes
- version control

I tried a few extensions myself, and it actually changed how the editor behaves. It almost feels like you are building your own customized tool.

This flexibility is what makes VS Code suitable for different types of developers.

4. Part D — Comparison

To understand things better, I compared Visual Studio Code with Sublime Text.
Instead of just listing features, I tried to think from a practical point of view.

VS Code:

- more features
- highly customizable
- completely free

Sublime Text:

- faster performance

- simpler interface
- paid license (after trial)

From my experience, VS Code feels more powerful because of its extensions and flexibility. It can be adapted for different use cases, which is useful in modern development.
However, Sublime Text still has its strengths. It is lightweight and fast, which makes it better for systems with limited resources or for users who prefer a minimal setup.
So I don't think one is completely better than the other.
It depends on what the user needs:

- flexibility and features → VS Code
- speed and simplicity → Sublime Text

6. Shell Scripts

Script 1: System Identity Report

Code:

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Ashwin Goyal | Course: Open Source Softw

# --- Variables ---
STUDENT_NAME="Ashwi Goyal    "
SOFTWARE_CHOICE="Visual Studio Code"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
HOME_DIR=$HOME
DISTRO=$(lsb_release -d 2>/dev/null | cut -f2)

# --- Display ---
```

```
echo "==========================================="
echo " Open Source Audit — $STUDENT_NAME" echo
"==========================================" echo
"Software Chosen : $SOFTWARE_CHOICE"
echo "Kernel Version : $KERNEL"
echo "User : $USER_NAME" echo
"Home Directory : $HOME_DIR" echo
"Uptime : $UPTIME" echo "Date &
Time : $DATE" echo "Distro :
$DISTRO"
echo "License : Linux OS is generally under GPL license"
echo "==========================================="
```

Output:

```
$ ../scrpit1.sh
./script1.sh: line11:eho Open Source Audit — Ashwin Goyal
===================================
Software Chosen  : Visual Studio Code
Kernel Version   : 3.6.4-b9f03e96.x86_64
User             : Ashwin Goyal
Home Directory   : /c/Users/Asus
Uptime           :
Date & Time      : Sat, Mar 28, 2026  6:45:09 PM
Distro           :
License          : Linux OS is generally under GPL license
===================================: No such file or directory
./script1.sh: line 11: No such file or directory
```

Explanation:

When I ran this script, it felt like a basic introduction to the system, almost like a welcome screen that tells you what environment you are working in.

It displays details such as:

- kernel version
- current username

- system uptime
- home directory
- date and time

At first, it looked very simple, and I didn't think much of it. But after looking more closely, I realized that it is actually pulling real-time system data using Linux commands like uname, whoami, and uptime.

This helped me understand how shell scripting can directly interact with the operating system and retrieve useful information.

Another thing I noticed is the use of variables and command substitution ($()). Instead of hardcoding values, the script dynamically fetches them every time it runs. That makes the output always accurate and up to date.

Overall, even though the script is simple, it gave me a good starting point to understand how scripts can automate system-level tasks.

Script 2: FOSS Package Inspector

Code:

```
#!/bin/bash # Script 2: FOSS Package Inspector (Clean Version)

PACKAGE="code"
echo "Checking package: $PACKAGE"
echo "-------------------------"

# Check if VS Code is installed if
command -v code &> /dev/null; then
echo "VS Code is installed."

# Show version
code --version | head -n 1
```

```
# Show path
echo "Location: $(which code)" else
echo "VS Code is NOT installed." fi

# Case statement
case $PACKAGE in
code) echo "VS Code: a lightweight, extensible open-source editor" ;;
git) echo "Git: version control system built for collaboration" ;; nginx)
echo "Nginx: high-performance web server" ;;
python) echo "Python: powerful open-source programming language" ;;
*) echo "Unknown package" ;;
esac
```

Output:

```
$ ./script2.sh
Checking package: code
--------------------------
VS Code is installed.
1.113.0
Location: /c/Users/Asus/AppData/Local/Programs/Microsoft VS Code/bin/code
VS Code: a lightweight, extensible open-source editor
```

Explanation:

This script is slightly more logical compared to the first one.

Its main purpose is to check whether Visual Studio Code is installed on the system or not. Instead of assuming anything, it actually verifies the presence of the code command using system checks.

If the software is installed, it shows:

- version details
- installation path

If not, it clearly prints that the software is not installed.

What I found interesting here is the use of conditional statements (if-else). It made me realize how scripts can make decisions based on system conditions.

The case statement part was also useful. Initially, I didn't fully understand why it was needed, but after running the script, it became clear that it helps in handling multiple software options and printing relevant descriptions.

This script shows how automation can be combined with logic, making it more practical than just displaying information.

Script 3: Disk and Permission Auditor

Code:

```
#!/bin/bash # Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"
# Loop through directories for
DIR in "${DIRS[@]}"; do if [
-d "$DIR" ]; then
PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

echo "$DIR => Permissions: $PERMS | Size: $SIZE" else
echo "$DIR does not exist"
fi done

echo "----------------------"
```

```
# VS Code config directory check
CONFIG_DIR="$HOME/.config/Code"

if [ -d "$CONFIG_DIR" ]; then
PERMS=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}') echo
"VS Code Config Found:"
echo "$CONFIG_DIR => $PERMS"
else
echo "VS Code config directory not found"
fi
```

Output:

```
$ ../script3.sh
Directory Audit Report
======================
./script3.sh: line 11: echo/etc  =>> Permissions: drwxr-xr-x Ashwin Goyal 197121 | Size: 1
/var/log does not exist
/home does not exist
/usr/bin => Permissions: drwxr-xr-x AshwinGoyal 197121 | Size: 87M
/tmp => Permissions: drwxr-xr-x AshwinGoyal 197121 | Size: 289M: No such file or directory
==========================
VS Code config directory found:
/home/frosfres/.config/Code => drwx -----frosfres frosfres
==========================
VS Code config directory found:
/home/frosfres/.config/Code
```

Explanation:

This script helped me understand the structure of Linux directories in a more practical way.

It loops through important system directories like:

- /etc
- /var/log
- /home
- /usr/bin

- /tmp

For each directory, it checks:

- permissions
- ownership
- disk usage

Before this, I had only theoretical knowledge about directory permissions. But seeing them displayed like this made it much clearer.

The use of a for loop was also important. Instead of checking each directory manually, the script automates the process and handles all directories in one go.

Another useful part is the check for the VS Code configuration directory (~/.config/Code). This connects the script directly to the chosen software, which makes it more relevant to the audit.

Overall, this script made me more comfortable with how Linux manages files, permissions, and storage.

Script 4: Log File Analyzer

Code:

```
#!/bin/bash # Script 4: Log File Analyzer

LOGFILE=$1 KEYWORD=${2:-"error"} COUNT=0
# Check file exists if [ ! -f "$LOGFILE"
]; then echo "Error: File $LOGFILE not
found." exit 1 fi

# If file empty → retry (simple loop)
while [ ! -s "$LOGFILE" ]; do echo
```

```
"File is empty. Waiting..." sleep 2
done

# Read file line by line while
IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
$ ../scrpit4.sh
Keyword 'error' found 6 times in/var/log/pacman.log
=====================================
Last 5 matching lines:
======================
[2026-03-09T16:50:55+0000] Installed perl-error (0.17030-3)
[2026-03-14T22:21:57+0530] Installed xcb-util-errors (1.0.1-2)
[2026-04-01T00:07:53+0530] Installed lib32-libgpg-error (1.58-1)
[2026-04-25T12:30:40+0530] Upgraded Libgpg-error (1.58-1 > 1.59-1 1)
[2026-05-03T12:30:43+0530] Upgraded Lib32-Libgpg-error (1.58-1 > 1.59-1)
=================================
./script4.sh: line 11: No such file or directory
```

Explanation:

This script felt more practical and closer to real-world usage.

It reads a log file and searches for a specific keyword (by default, “error”). Then it counts how many times that keyword appears.

What I found useful is how it processes the file line by line using a while loop. This shows how scripts can handle large amounts of data efficiently.

It also uses grep to filter matching lines and tail to display the last few results. This makes the output more focused and useful instead of overwhelming.

Another interesting feature is that it checks if the file exists and whether it is empty before processing it. This kind of validation is important in real applications.

I can actually see this script being useful for debugging issues, especially when dealing with large log files where manually searching would take too much time.

Script 5: The Open Source Manifesto Generator

Code:

```
#!/bin/bash # Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

# Taking input read -p "1. One open-source tool you use
daily: " TOOL read -p "2. One word for freedom: "
FREEDOM read -p "3. One thing you would build: "
BUILD

DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"

# Creating manifesto echo "Open Source
Manifesto" > $OUTPUT echo "Date:
$DATE" >> $OUTPUT echo "" >>
$OUTPUT
echo "I use $TOOL every day, which represents the power of open
collaboration." >> $OUTPUT
```

```
echo "For me, freedom means $FREEDOM in the digital world." >> $OUTPUT
echo "In the future, I aim to build $BUILD and share it openly." >> $OUTPUT
echo "I believe open source drives innovation and collective growth." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo "-------------------------"
cat $OUTPUT
```

Output:

```
$ ../scrpit5.sh
./script5.sh: line 11:eho Open Source Audit — Ashwin Goyal
==========================================
Answer three questions to generate your manifesto.
1. One open-source tool you use daily: VS code
2. One word for freedom: Modi
3. One thing you would build: developer tools

Manifesto saved to manifesto_Ashwin.txt
----------------------
Open Source Manifesto
Date: 29 March 2026

I use VS code every day, which represents the power of open collaboration.
For me, freedom means Modi in the digital world.
In the future, I aim to build developer tools and share it openly.
I believe open source drives innovation and collective growth.

./script5.sh: line 11: No such file or directory
```

Explanation:

This was the most interesting one.
Instead of just running commands, it interacts with the user.
It asks questions and builds a small “manifesto.”
This shows how scripting is not just technical — it can also be creative.

Open Source vs Proprietary Comparison Visual Studio Code vs Sublime Text

Feature	Visual Studio Code (Open Source)	Sublime Text (Proprietary)
Cost	Free	Paid (trial available)
Source	Open (Code-OSS) Code	Closed
Security	Can be audited by anyone	Cannot be inspected
Customization	Highly customizable via extensions	Limited customization
Extensions	Huge marketplace	Smaller ecosystem
Performance	Slightly heavier (Electron-based)	Very fast and lightweight
Updates	Frequent community-driven updates	Controlled by company
Support	Community + documentation	Official support available
Control	Community + Microsoft	Fully company controlled

Visual Studio Code and Sublime Text are both tools used for writing and editing code, but after comparing them closely, I realized they are actually quite different in how they are built and used.

VS Code is based on an open-source model (at least its core version, Code-OSS). This means developers are able to see how the software works internally, modify it, and even contribute to its development. Earlier, I didn't think this was a big deal, but while working on this assignment, I started to understand why it matters. When software is open, it builds a sense of trust because nothing is hidden. If needed, developers can fix bugs themselves or improve features instead of waiting for a company.

On the other hand, Sublime Text is proprietary. It is developed and maintained by a single company, and users don't have access to its source code. For basic usage, this might not feel like a problem, but it does limit flexibility at a deeper level. You are dependent on the company for updates and improvements.

Another major difference is cost. VS Code is completely free, which makes it very accessible, especially for students and beginners. Anyone can download it and start using it without any restrictions. Sublime Text provides a free trial, but eventually asks users to purchase a license. While this might not matter much for professionals, it can be a limitation for learners or people who are just starting out.

However, while comparing both tools, I noticed that Sublime Text has its own advantages. It is known for being extremely fast and lightweight. Since it does not rely on heavy frameworks like Electron (which VS Code uses), it runs more smoothly, especially on systems with limited resources. Even simple actions like opening files or switching between tabs feel quicker.

VS Code, on the other hand, can sometimes feel slightly heavier. This is mainly because it is built using web technologies like Electron and Chromium. Initially, I thought this was a disadvantage, but later I realized that this design also allows it to support a wide range of features and extensions.

One of the biggest strengths of VS Code is its extension ecosystem. This is something that really stands out. You can install extensions for almost anything — programming languages, debugging tools, themes, version control, and more. Because of this, VS Code can be customized into a full development environment based on your needs.

Sublime Text also supports plugins, but its ecosystem is smaller and not as active as VS Code's. As a result, VS Code feels more suitable for modern development where integration with multiple tools is important.

Another interesting difference is the idea of control and development style. VS Code is influenced by both Microsoft and the open-source community. So it is not fully independent, but it is also not completely controlled by a company. This creates a mixed model where both corporate and community contributions exist.

Sublime Text, in contrast, is entirely controlled by its developers. This makes decision-making more centralized, but also less flexible from a community perspective.

From my point of view, both tools have their place. VS Code is more powerful and flexible, especially for modern workflows. It supports multiple languages, tools, and extensions, which makes it suitable for a wide range of use cases.

At the same time, Sublime Text is still a strong option for users who prefer simplicity and speed. Some developers might not need all the extra features of VS Code and may prefer a cleaner, faster editor.

So instead of saying one is better than the other, I think it depends on the situation:

- If someone wants flexibility and features → VS Code
- If someone wants speed and simplicity → Sublime Text

Conclusion:

This audit changed my understanding of open source.
Earlier, I saw it as free software. Now I see it as a system built on trust, collaboration, and shared growth.
Visual Studio Code is a good example of how open source and corporate involvement can work together — but it also shows that balance is important.
In the end, I feel open source is not just about code — it is about people and how they work together.